성장 과정 및 직무 선택 동기

[0.1초의 인터랙션까지 설계하는, 디자인하는 개발자]
산업디자인을 전공하며 느꼈던 가장 큰 갈증은 '정적인 시안'만으로는 '살아 움직이는 서비스'의 진짜 경험을 완전히 구현할 수 없다는 점이었습니다. 아무리 세련된 인터페이스를 만들어도, 데이터가 잠깐이라도 지연될 때 매끄럽게 안내하지 않거나, 예상치 못한 상황에 친절하게 대응하지 못하면 진정한 사용자 경험과는 거리가 있다는 사실을 절실히 깨달았습니다. 그래서 저는 사용자가 체감하는 0.1초의 미묘한 인터랙션까지 직접 구현하고 싶다는 열망을 품게 되었고, 결국 프론트엔드 개발자로 방향을 틀게 되었습니다.

단순한 관심을 넘어서 실제 역량을 갖추기 위해 논리적 기반을 다져나갔습니다. 정보처리기사와 SQLD 자격증을 준비하고 취득하면서 컴퓨터공학의 기본기를 차곡차곡 쌓았고, 프로젝트에서는 Figma로 만든 UI 요소들을 독립적인 컴포넌트로 표준화하여 코드의 재사용성과 팀 협업 효율을 모두 높일 수 있는 저만의 프로세스를 구축했습니다. 이러한 꼼꼼함과 집요함은 곧 실질적인 성과로 이어졌습니다. 예를 들어, 'BookDam' 프로젝트에서는 화면 구현을 넘어 계층형 댓글 구조를 직접 설계하고 적용해 데이터 무결성을 지켜냈으며, 'CafeOn' 프로젝트에서는 React Query와 Zustand를 활용해 데이터 로딩과 에러 처리 과정을 한층 정교하게 다듬었습니다. 덕분에 불안정한 네트워크 환경에서도 사용자가 자연스럽고 신뢰감 있게 서비스를 이용할 수 있도록 개선할 수 있었습니다.

저는 0.1초의 미묘한 차이, 그리고 보이지 않는 예외 처리들이 결국 서비스의 완성도를 좌우한다고 믿습니다. 사용자 관점에서 생각하는 디자인 감각을 논리적인 코드로 녹여내어, 기획의 의도를 100% 사용자에게 전달할 수 있는 프론트엔드 개발자가 되겠습니다.

팀 프로젝트 협업 및 갈등 극복

[예상치 못한 변수 속에서 소통으로 균형을 잡다.]
첫 팀 프로젝트인 'BookDam'에서 초기 역할 분담 과정의 소통 오류로 인해, 마이페이지 개발 업무가 갑자기 제게 넘어오는 상황이 발생했습니다. 당시에는 이미 커뮤니티 API 구현을 책임지고 있어 일정이 빠듯했기 때문에, 자칫하면 프로젝트 전체 완성도가 떨어질 수 있는 위기였습니다.

이런 혼란스러운 상황에서도 저는 감정적으로 대응하기보다는 객관적으로 현황을 공유하고 우선순위를 재조정하는 데 집중했습니다. 우선, 팀원들에게 현재 맡고 있는 업무량과 예상 완료 시점을 솔직하게 알렸고, 이를 통해 모두가 프로젝트 리스크를 정확히 인식할 수 있도록 했습니다. 그리고 진행하던 커뮤니티 페이지의 핵심 기능인 계층형 댓글 구현을 최우선 과제로 삼으면서, 동시에 새롭게 추가된 마이페이지 업무는 Prisma Schema를 미리 정의하는 방식으로 데이터 모델링의 반복 작업을 줄여 효율을 높였습니다.

그 결과, 두 가지 주요 기능을 모두 기한 내에 안정적으로 완수할 수 있었고, 이 과정에서 철저한 명세서 작성과 신속한 변경사항 공유가 얼마나 중요한지 직접 깨달았습니다. 이후 'CafeOn' 프로젝트에서는 이러한 경험을 바탕으로, Notion과 Slack을 활용한 실시간 업무 공유 프로세스를 확립했습니다. 그 덕분에 프론트엔드 담당자로서 5개 이상의 주요 페이지—실시간 채팅부터 알림 시스템까지—를 충돌 없이 안정적으로 개발할 수 있었습니다. 예상치 못한 변수가 닥쳤을 때에도 당황하지 않고, 데이터와 소통을 통해 팀 전체의 균형을 지켜내는 개발자가 되겠습니다.

성격의 장단점

[설계의 신중함: 안정적인 결과물과 유연한 수용 사이의 균형]
저의 가장 큰 강점은 '선 설계, 후 구현'의 원칙 아래, 구현의 가능성을 다각도로 치밀하게 검토하는 신중함입니다. 산업디자인 전공 경험을 바탕으로, Figma로 설계를 시작할 때부터 예외 상황과 반응형 그리드를 한 발 앞서 점검하며 UI 요소들을 컴포넌트 단위로 표준화합니다. 이런 꼼꼼한 접근 덕분에 실제 개발 과정에서 레이아웃 수정에 소요되는 시간을 30% 이상 절감할 수 있었고, "함께 작업하기 든든하다"는 신뢰를 동료들에게 얻었습니다.

물론 이러한 신중함이 늘 장점으로만 작용하는 것은 아닙니다. 검증된 방식에 안주하며 새로운 기술 도입에는 신중하다보니, 때때로 변화에 다소 보수적인 선택을 할 수도 있다는 점을 스스로 경계하고 있습니다. 지나치게 안정성만을 좇다 보면 성장의 기회를 놓칠 수 있기 때문입니다.

이러한 한계를 보완하기 위해 저는 '근거 있는 유연함'을 실천하고자 노력합니다. 무턱대고 새로운 기술을 받아들이거나 반대로 막연하게 거부하는 대신, 도입하고자 하는 도구의 기대 효과와 리스크를 스스로 분석하여 '검토 문서'로 정리합니다. 이런 과정을 통해 기술적 호기심도 꾸준히 키우고 있습니다. 실제로 팀 프로젝트에서 Swagger나 새로운 상태 관리 라이브러리 도입 제안이 들어왔을 때, 먼저 자료를 찾아보고 취지와 영향도를 정리해 팀원들과 소통하였습니다. 변화 앞에서 주도적으로 의견을 제시함으로써, 새로운 도전을 성장의 동력으로 삼는 습관을 만들고 있습니다.

이처럼 저는 신중함이라는 기본기를 '안정적인 설계'의 강점으로 살리면서, 철저한 분석을 바탕으로 변화에도 유연하게 대응할 수 있는 개발자, 즉 흔들리지 않으면서도 열려 있는 전문가로 성장하겠습니다.